

SIFT1M VERİ KÜMESİ ÜZERİNDE BÜYÜK ÖLÇEKLİ VEKTÖR ARAMA ANALİZİ

1. Giriş ve Amaç

Bu proje, modern yapay zekâ ve arama motoru sistemlerinin kalbi olan Vektör Benzerlik Araması (Vector Similarity Search) yöntemlerini incelemek amacıyla gerçekleştirilmiştir. 1 Milyon adet 128 boyutlu SIFT vektörü üzerinde; kesin sonuç veren Brute-Force yöntemi ile yüksek performanslı Approximate Nearest Neighbor (ANN) algoritmalarının performans karşılaştırması hedeflenmiştir.

2. Veri Seti Teknik Özellikleri: SIFT1M

- Veri Tipi:** SIFT (Scale-Invariant Feature Transform) tanımlayıcıları.
- Boyut:** 1.000.000*128 (1 Milyon vektör, her biri 128 boyutlu).
- Mesafe Metriği:** L2 (Öklid Mesafesi).
- İçerik:** base (ana veritabanı,xb), query (sorgu seti,xq), learn (eğitim seti,xt) ve ground truth (referans doğru sonuçlar) olmak üzere 4 ana bölümden oluşmaktadır.

3. İndeks Yapılarının Karşılaştırmalı Analizi

A. IndexFlatL2 (Referans Noktası)

- Karakteristiği:** "Kaba Kuvvet" (Brute-Force) yaklaşımıdır.
- Analiz:** Herhangi bir indeks yapısı kurmadığı için **İnşa Süresi en düşüktür**. Ancak arama yaparken her vektörü tek tek taradığı için **Sorgu Gecikmesi (Latency) en yüksektir**.
- Kullanım Yeri:** %100 doğruluk gerektiren küçük veri setleri veya ANN algoritmalarını test etmek için "Ground Truth" oluşturma.

B. IndexIVFFlat (Inverted File Index)

-Karakteristiği: IndexIVFFlat, veri uzayını birbirine benzeyen vektörlerin toplandığı hücelere (cluster) ayırarak çalışan bir yapıdır. FlatL2 indeksinin aksine, her sorguda tüm veri kümesini taramak yerine yalnızca ilgili hücreler üzerinde arama yaparak sorgu süresini önemli ölçüde azaltır.

1. nlist ve Cluster Yapılandırması

Bu indekste en kritik hiperparametre, toplam hücre sayısını belirleyen **nlist** değeridir. FAISS dokümantasyonu bu parametre için kesin bir kural önermemekle birlikte, vektör arama literatüründe yaygın olarak kullanılan bir heuristic yaklaşım olan $\sqrt{N}$ (karekök kuralı) tercih edilmiştir.

SIFT1M veri seti için:

$$N=1.000.000 \rightarrow \sqrt{N} \rightarrow 1000$$

Bu doğrultuda:

$$*nlist = 1000$$

Bu seçim, hücre merkezlerini bulma maliyeti ile hücre içi arama maliyeti arasında dengeli bir trade-off sağlayarak hem performans hem de doğruluk açısından verimli sonuçlar üretir.

2. nprobe ve Hız-Doğruluk Dengesi (Trade-off)

nprobe parametresi, bir sorgu sırasında kaç adet hücrenin taranacağını belirleyen kritik bir parametredir.

- **Düşük nprobe (örn: 1):**

Sadece en yakın hücre incelenir. Bu durumda arama çok hızlıdır ancak doğru sonucun komşu hücrelerde bulunması halinde kaçırılma ihtimali yüksektir (recall düşer).

- **Yüksek nprobe (örn: 100):**

Daha fazla hücre tarandığı için doğru sonuca ulaşma olasılığı (recall)

önemli ölçüde artar. Ancak buna karşılık sorgu gecikmesi (latency) de artar.

nprobe	Latency (ms)	Recall@10
1	0.0229	0.4651
5	0.1084	0.8206
10	0.2101	0.9163
50	0.9904	0.9962
100	1.9010	0.9993

C. IndexHNSW (Hierarchical Navigable Small World)

-Karakteristiği: Vektörler arası hiyerarşik bir grafik yapısıdır.

-Analiz: Graf yapısını kurmak için çok sayıda bağlantı hesapladığından **İnşa Süresi en yüksektir**. Ancak arama anında graf üzerinde "atlayarak" ilerlediği için **Sorgu Gecikmesi en düşüktür**.

-Kullanım Yeri: Milisaniyelik yanıt süresinin kritik olduğu üretim sistemleri.

4. Performans Metrikleri ve Mühendislik Çıkarımları

I. İnşa Süresi (Build Time) vs. Sorgu Süresi (Latency) İlişkisi:

Rapordaki en kritik ayrım şudur: Bir indeksin **İnşa Süresinin fazla olması**, onun **Sorgu Gecikmesinin az olacağı** anlamına gelir.

HNSW'de biz "peşin ödeme" yaparız: İndeksi kurarken (Build Time) bilgisayarı çok yorarız (Graf yapısı oluştururuz), ancak bu zahmet sayesinde arama yaparken (Latency) yolu çok kısaltırız.

II. Recall@10 Analizi

ANN algoritmaları "yaklaşık" sonuçlar verir. Recall@10 = 0.95 demek; "Sorguladığım 100 verinin 95'inde, gerçek en yakın komşuyu ilk 10 sonuç içerisinde bulabildim" demektir.

III. Bellek (RAM) Kullanımı ve Sistem Dinamikleri

Vektör indeksleme süreçlerinde bellek kullanımı yalnızca indeksin yapısına değil, aynı zamanda Python çalışma zamanı ve işletim sisteminin bellek yönetim mekanizmalarına da bağlıdır.

- **FlatL2:** Ölçümlerde bellek kullanımında anlamlı bir artış gözlemlenmemiştir. Bu durum, indeksin ek bir veri yapısı oluşturmadan vektörleri doğrudan saklamasından kaynaklanmaktadır. Bellek kullanımı büyük ölçüde veri setinin kendisi tarafından belirlenmektedir.
- **IVFFlat:** Bu indeks, veri setine ek olarak cluster yapıları tuttuğu için teorik olarak FlatL2'ye kıyasla daha fazla bellek kullanımı gerektirir. Ancak ölçümlerde gözlemlenen küçük dalgalanmalar, sistem seviyesindeki bellek yönetimi etkilerinden kaynaklanabilir.
- **HNSW:** Grafik tabanlı yapısı nedeniyle her vektör için komşuluk bağlantılarını saklar ve bu nedenle teorik olarak en yüksek bellek kullanımına sahiptir. Bununla birlikte, inşa süreci sonrası RAM değerlerinde gözlemlenen düşüşler; **garbage collection (çöp toplama)**, bellek yeniden tahsisi (reallocation) ve işletim sistemi bellek yönetimi (paging) gibi faktörlerden kaynaklanabilir.

-Mühendislik Çıkarımı: Elde edilen RAM ölçümleri, indekslerin kesin bellek maliyetinden ziyade sistemin çalışma anındaki toplam bellek davranışını yansıtmaktadır. Bu nedenle, bellek analizleri yapılırken ölçüm ortamı ve sistem dinamikleri mutlaka göz önünde bulundurulmalıdır.

PERFORMANS ÖZET TABLOSU						
Index Tipi	İnşa Süresi (s)	Ortalama Latency (s)	Recall@10	RAM Kullanımı(Önce)	RAM Kullanımı(Sonra)	
FlatL2	0.294868	0.001935	1.0000	1413.687500	1865.523438	
IVFFlat	3.412550	0.000211	0.9163	1882.984375	2541.582031	
HNSW	55.189529	0.000031	0.9109	2559.578125	2474.875000	

5. Pareto Analizi: Hangi Senaryoda Hangi İndeks?

Pareto optimizasyonuna göre, kullanım senaryoları şu şekildedir:

1.Senaryo - "Hız Her Şeydir": (Örn: Google Görseller, Canlı Öneri Sistemleri)

Seçim: IndexHNSW

Neden: Yüksek inşa maliyetine rağmen, kullanıcının beklediği o milisaniyelik yanıtı sadece bu yapı verebilir.

2.Senaryo - "RAM Kısıtlı / Orta Hız": (Örn: Kısıtlı bütçeli sunucular)

Seçim: IndexIVFFlat

Neden: Bellek kullanımı HNSW'den çok daha düşüktür ve nprobe ile ihtiyaca göre hız/doğruluk ayarı yapılabilir.

3.Senaryo - "Asla Hata İstemiyorum": (Örn: Adli tıp veri eşleme, akademik araştırmalar)

Seçim: IndexFlatL2

Neden: Zaman önemli değildir, önemli olan tek şey %100 doğru sonuçtur.

6.Genel Değerlendirme ve Sonuç

Bu çalışma, SIFT1M veri kümesi üzerinde Hız–Hafıza–Doğruluk dengesi açısından farklı indeks yapılarının performansını analiz etmiştir. Elde edilen sonuçlar aşağıda özetlenmiştir:

- **ANN yöntemleri zorunludur:** 1 milyon vektör ölçeğinde IndexFlatL2 (brute-force) yaklaşımı kabul edilemez sorgu gecikmesi üretmekte, bu nedenle ölçeklenebilir sistemlerde ANN yöntemleri temel gereklilik haline gelmektedir.
- **İnşa–sorgu dengesi:** IndexHNSW, yüksek inşa maliyetine rağmen sorgu aşamasında sağladığı düşük gecikme ile en yüksek performansı sunmaktadır. Bu durum, “ön hesaplama ile sorgu optimizasyonu” prensibini doğrulamaktadır.
- **Esnek yapı:** IndexIVFFlat, nlist ve özellikle nprobe parametresi sayesinde doğruluk ve hız arasında ayarlanabilir bir denge sunarak orta seviye sistemler için uygun bir alternatif oluşturmaktadır.

Nihai sonuç olarak, büyük ölçekli veri sistemlerinde düşük gecikme gereksinimi olan uygulamalar için IndexHNSW en uygun seçenekken, kaynak kısıtlı veya dengeli sistemlerde IndexIVFFlat tercih edilmelidir. IndexFlatL2 ise yalnızca referans (ground truth) üretimi için uygundur.